CAMPUS CAFÉ ORDERING SYSTEM

Java OOP Laboratory Exercise

Names: Romer Enardecido & Bernard Russel

Section: E

Date: 10/11/2026

Submitted as a demonstration of classes, objects, constructors, encapsulation, inheritance, polymorphism,

abstraction, and conditional statements.

1. OOP Concept Explanations

Classes and Objects: The program has seven classes: MenuItem, Beverage, Pastry, Meal, Customer,

Order, and CafeApp. Objects are created in CafeApp to represent customers, food items, and orders.

Attributes and Behaviors: Each class has fields that store information and methods that perform actions.

For example, Beverage stores its size and computes its price.

Object Interaction: Order stores a Customer and up to three MenuItem references. It calls item methods

to compute prices and updates the customer's loyalty points after payment.

Abstraction: MenuItem is an abstract class with the abstract computePrice() method. Each concrete

menu-item subclass provides its own implementation.

Inheritance: Beverage, Pastry, and Meal extend MenuItem, so they reuse its common name, base price,

item code, and methods.

Encapsulation: Fields are private or protected and are accessed through getters and setters. Setters

validate values such as negative prices and negative extra-side counts.

Polymorphism and Overriding: Order stores different item types using MenuItem references. Calls to

computePrice() and getDescription() use the subclass implementations at runtime.

Method Overloading: Customer has addPoints(int) and addPoints(double). Order has computeTotal()

and computeTotal(double), allowing different parameter lists.

Conditional Statements: If and if-else statements validate data, determine item prices, enforce the

three-item order limit, and apply the student discount.

2. Program Output

The following is the actual console output produced after compiling and running the submitted Java files:

Order is full. Cannot add Cookie.

Error: base price cannot be negative.

\nOrder #1001 — Romer Enardecido (Student)

Iced Latte [B-01] Size: L 120.00

Ensaymada [P-01] (Warmed) 55.00

Chicken Adobo [M-01] Upsized, 1 side 142.00

Subtotal: 317.00

Student discount (10%): -31.70

TOTAL: 285.30

Loyalty points: 5

Error: base price cannot be negative.

\nOrder #1002 — Bernard Russel

Hot Chocolate [B-02] Size: M 95.00

Beef Tapa [M-02] Regular, 0 sides 130.00

Subtotal: 225.00

Voucher: -20.00

TOTAL: 205.00

Loyalty points: 4

3. Challenges Encountered and Solutions

One challenge was designing the Order class so it could store different menu-item types without using

arrays or collection classes. This was solved by creating three separate MenuItem fields and checking

each field before adding an item. Another challenge was making sure each item had its own pricing rule.

This was solved by declaring computePrice() as an abstract method in MenuItem and overriding it in

Beverage, Pastry, and Meal. Input validation was handled through setters so invalid negative values are

rejected while valid data is preserved.

4. How to Compile and Run

javac *.java\njava CafeApp

5. Screenshot Requirement

The console output above is included as text for reference. Before submission, open the project in a

terminal, run the two commands shown above, and insert a screenshot of your own program output here if

your instructor specifically requires an image screenshot.